RePr: Improved Training of Convolutional Filters

Aaditya Prakash, James Storer, Dinei Florencio and Cha Zhang

Presented by Niu Shuaicheng
Scut Machine Intelligence Laboratory

March 24, 2019

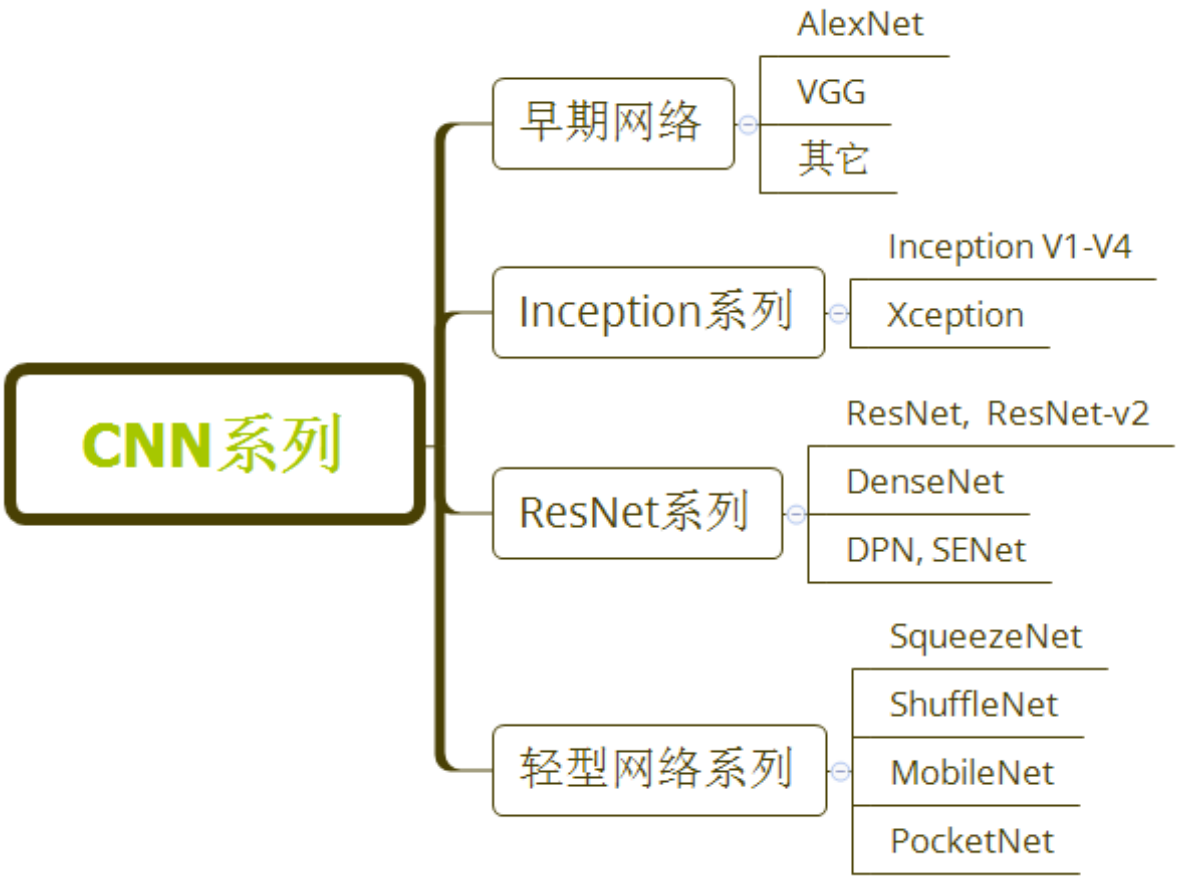
Outline

1. Background
2. Related Work
3. Motivation

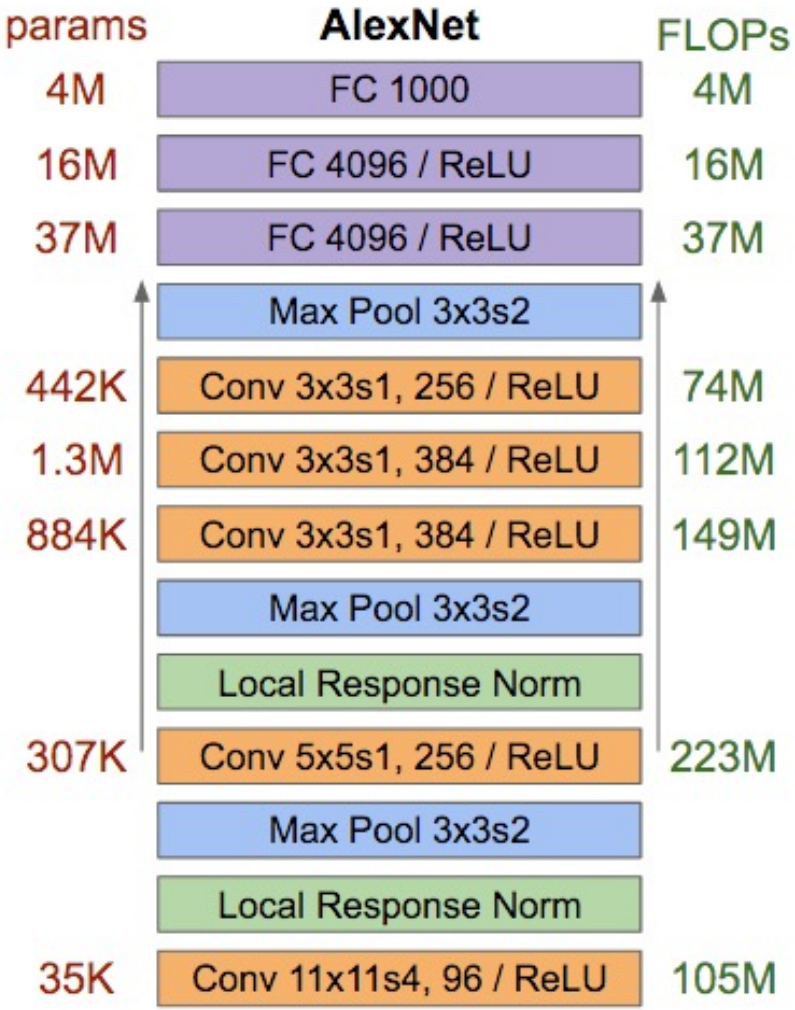
Outline

1. Background
2. Related Work
3. Motivation

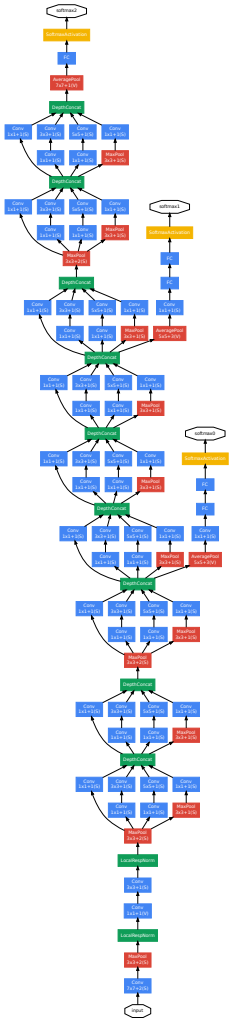
Convolutional Neural Networks



AlexNet: ImageNet Classification with Deep Convolutional Neural Networks



GoogLeNet: Going deeper with convolutions



Motivation

How to design neural network architectures automatically?

Outline

1. Background
2. Related Work
3. Motivation

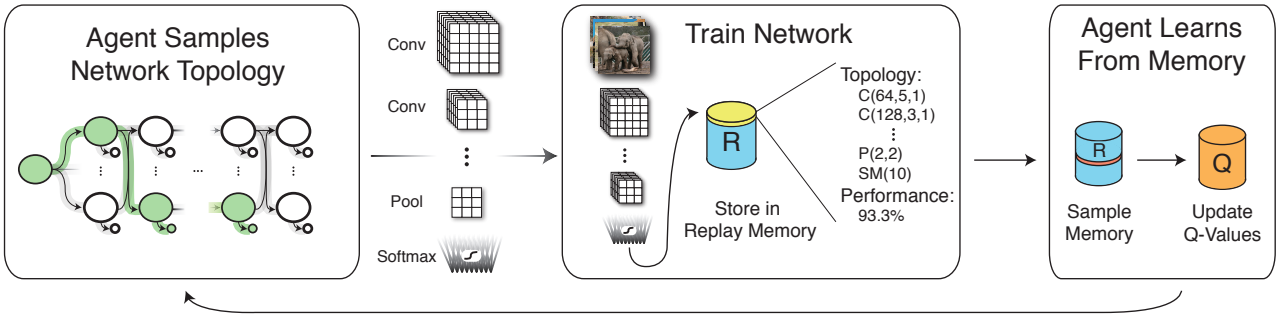
Related Work

Categorized by search methods:

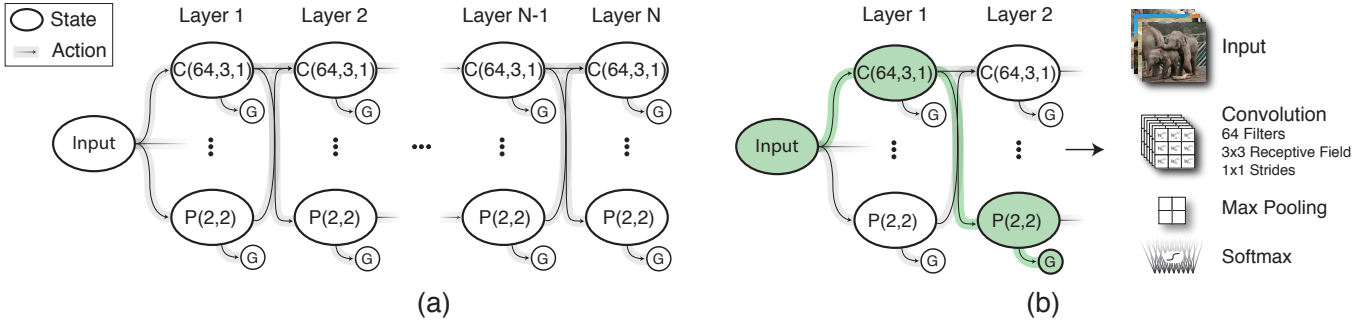
- Reinforcement learning (RL)
- Evolutionary algorithms (EA)
- Sequential model-based optimization (SMBO)
- Gradient ascent (GA)

The Workflow of MetaQNN

Framework:



Generation:



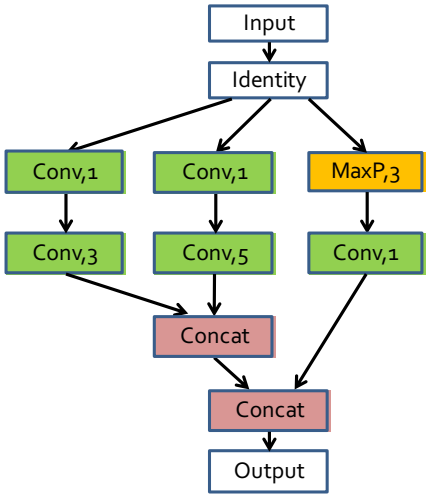
Experimental State Space

Layer Type	Layer Parameters	Parameter Values
Convolution (C)	$i \sim$ Layer depth $f \sim$ Receptive field size $\ell \sim$ Stride $d \sim$ # receptive fields $n \sim$ Representation size	< 12 Square. $\in \{1, 3, 5\}$ Square. Always equal to 1 $\in \{64, 128, 256, 512\}$ $\in \{(\infty, 8], (8, 4], (4, 1]\}$
Pooling (P)	$i \sim$ Layer depth $(f, \ell) \sim$ (Receptive field size, Strides) $n \sim$ Representation size	< 12 Square. $\in \{(5, 3), (3, 2), (2, 2)\}$ $\in \{(\infty, 8], (8, 4]$ and $(4, 1]\}$
Fully Connected (FC)	$i \sim$ Layer depth $n \sim$ # consecutive FC layers $d \sim$ # neurons	< 12 < 3 $\in \{512, 256, 128\}$
Termination State	$s \sim$ Previous State $t \sim$ Type	Global Avg. Pooling/Softmax

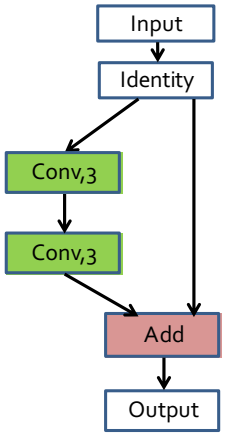
10 GPUs $\times$ 9-10 days on each dataset

Advanced Network Structure Code Space

Name	Index	Type	Kernel Size	Pred1	Pred2
Convolution	**T**	1	1, 3, 5	**K**	0
Max Pooling	**T**	2	1, 3	**K**	0
Average Pooling	**T**	3	1, 3	**K**	0
Identity	**T**	4	0	**K**	0
Elemental Add	**T**	5	0	**K**	**K**
Concat	**T**	6	0	**K**	**K**
Terminal	**T**	7	0	0	0



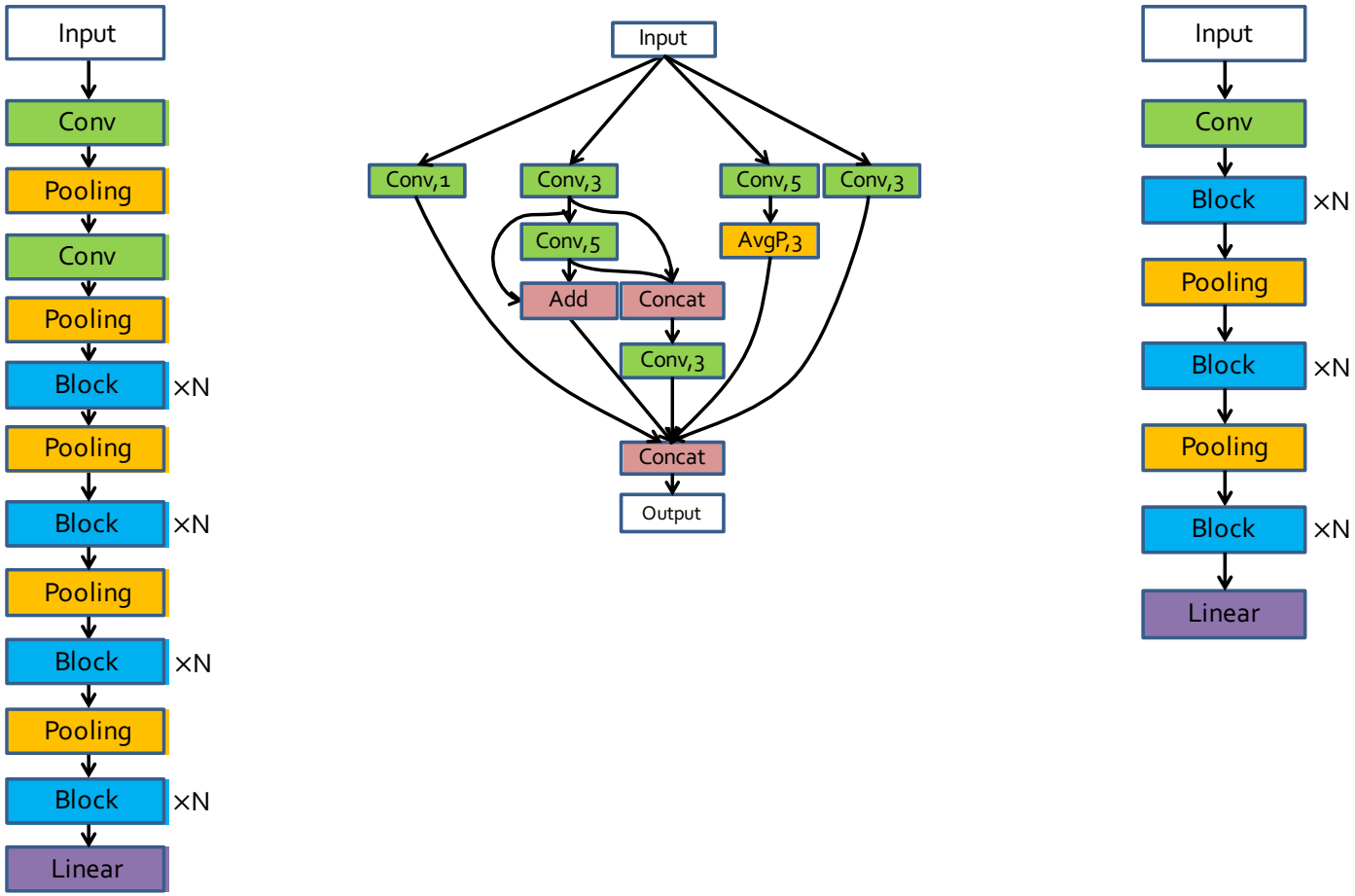
Codes = [(1,4,0,0,0), (2,1,1,1,0), (3,1,3,2,0), (4,1,1,1,0), (5,1,5,4,0), (6,6,0,3,5), (7,2,3,1,0), (8,1,1,7,0), (9,6,0,6,8), (10,7,0,0,0)]



Codes = [(1,4,0,0,0), (2,1,3,1,0), (3,1,3,2,0), (4,5,0,1,3), (5,7,0,0,0)]

32 GPUs $\times$ 3 days

Result of BlockQNN



Using a RNN as Architecture Generator

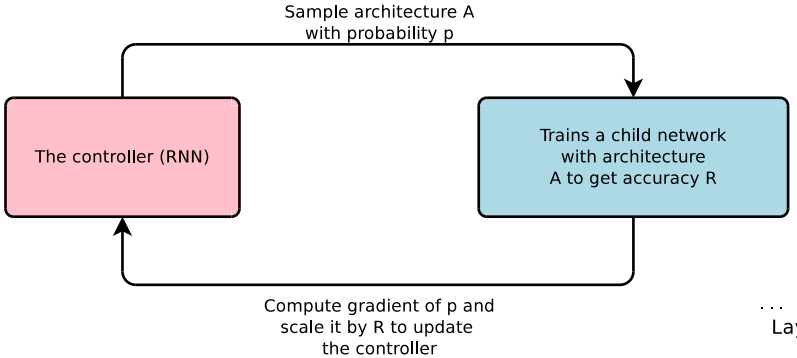


Figure 1: An overview of Neural Architecture Search.

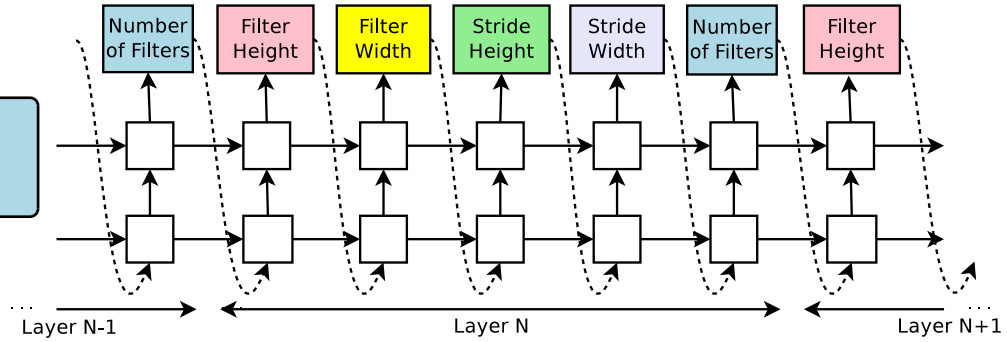
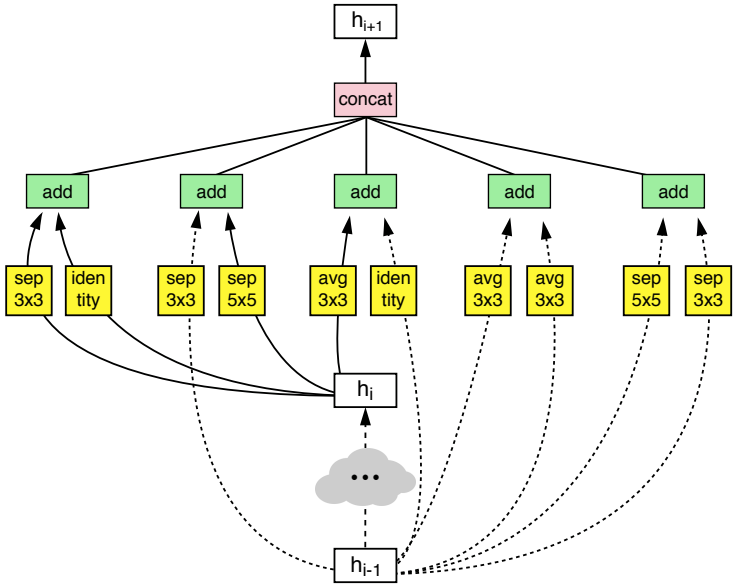


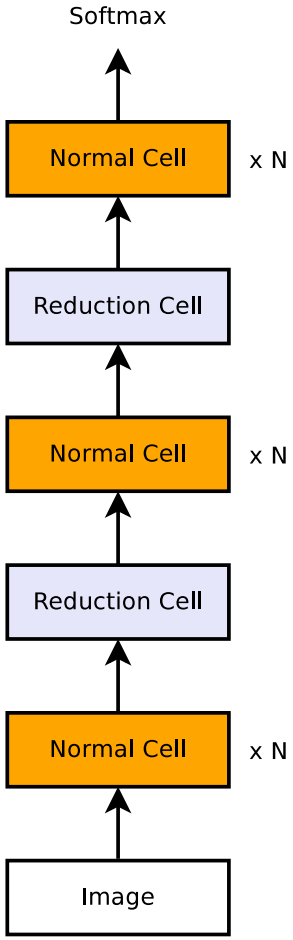
Figure 2: How the controller recurrent neural network samples a simple convolutional network.

800 GPUs $\times$ 28 days

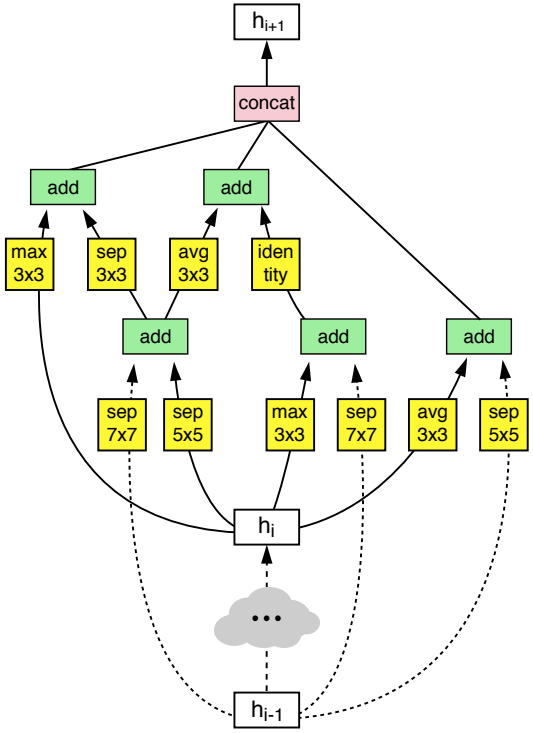
NASNet on CIFAR-10



Normal Cell



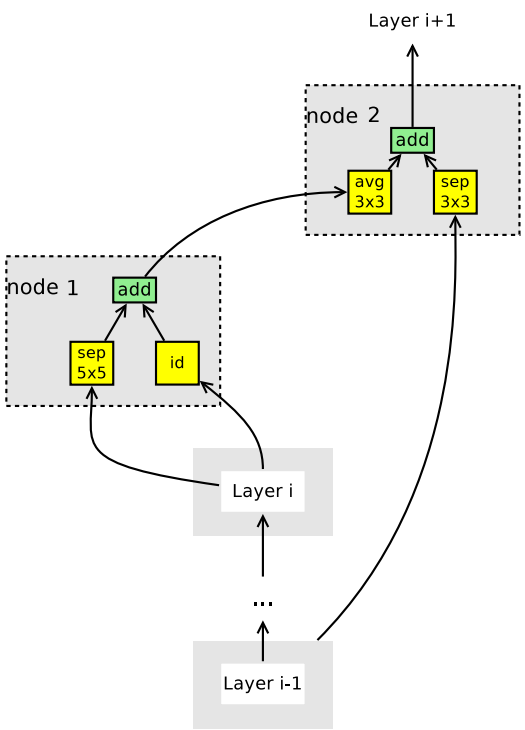
CIFAR10
Architecture



Reduction Cell

Cell Topologies of NASNet

The cell can be represented by a DAG consisting of B nodes. Each node is a mapping from 2 input tensors to 1 output tensor and can be specified as a 4-tuple.

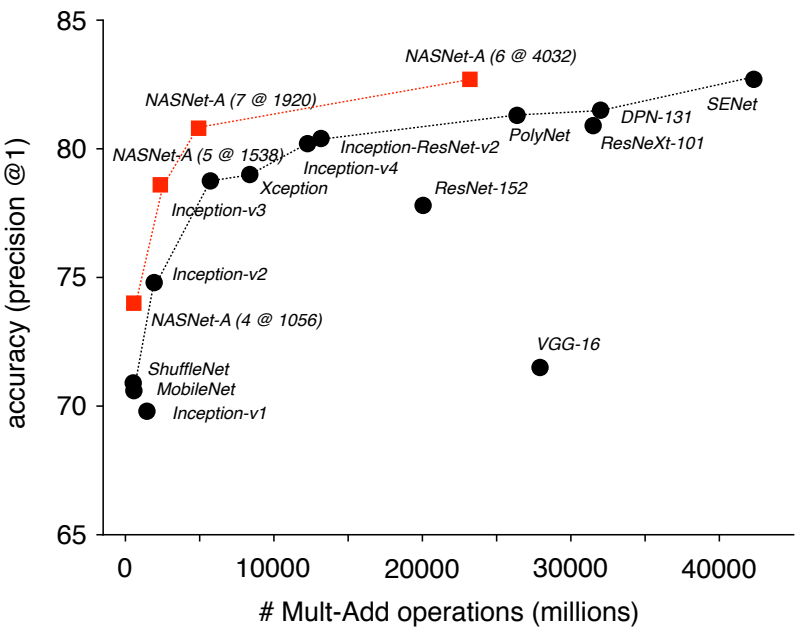


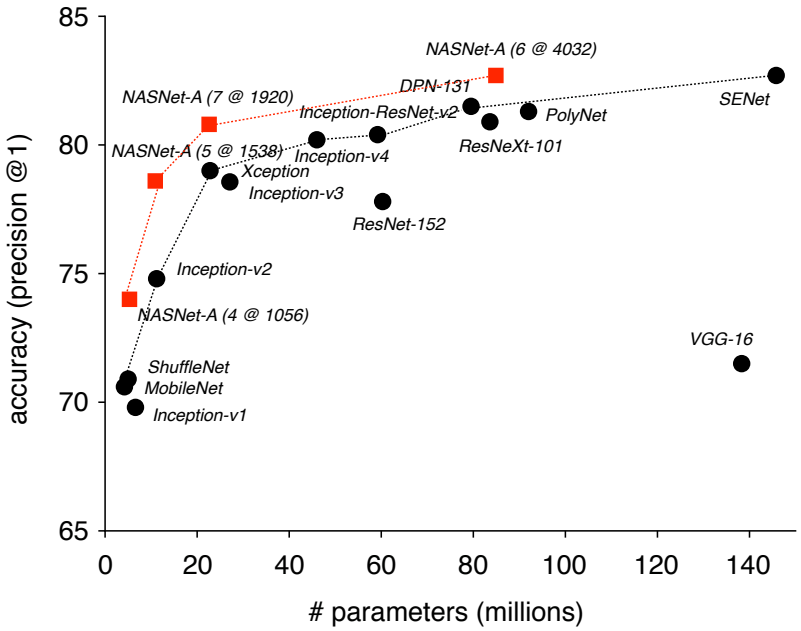
Definition

Node tuple in CNN cell: $< I_1, I_2, O_1, O_2 >$

- $I_1, I_2 \in \mathcal{I}_b$ specifies the inputs to the node
- $O_1, O_2 \in \mathcal{O}$ specifies the operation to apply to input I_i
- $\mathcal{I}_b = \{H_1^c, ..., H_{b-1}^c, H_B^{c-1}, H_B^{c-2}\}$ is the set of outputs of all previous nodes in this cell and the outputs of two previous cell
- $\mathcal{O}$ is the set of 11 functions, each of which operates on a single tensor

Performance on ImageNet





500 GPUs $\times$ 4 days

Parameter Sharing Technology

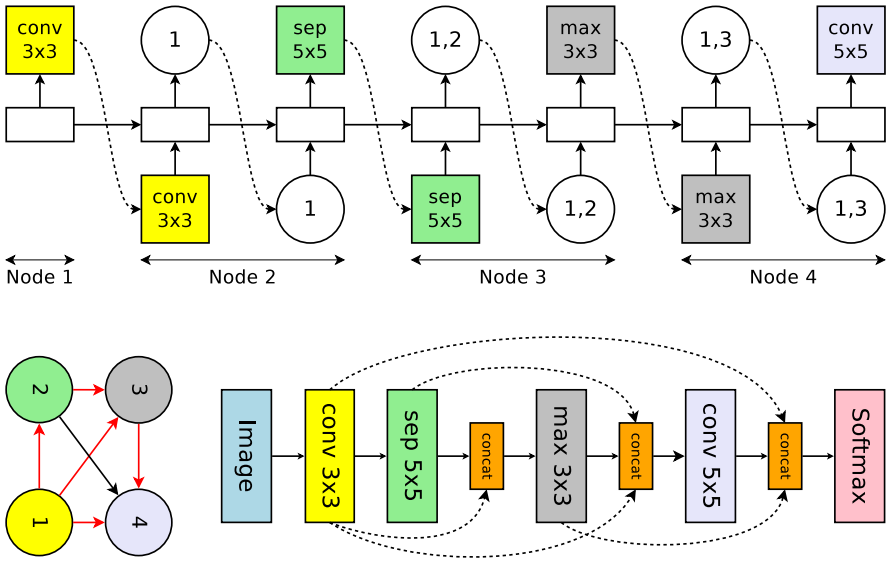


Figure 1. An example run of a recurrent cell in our search space with 4 computational nodes, which represent 4 layers in a con-volutional network. *Top:* The output of the controller RNN. *Bot-tom Left:* The computational DAG corresponding to the network's architecture. Red arrows denote the active computational paths. *Bottom Right:* The complete network. Dotted arrows denote skip connections.

1 GPU $\times$ 0.45 day

Formally, an architecture is defined by the representation *(G,* $\mathbf{o}$*)*, consisting of two ingredients:

- A set of available operations $\mathbf{o} = \{o_1, o_2, ...\}$
- An adjacency matrix G specifying the neural network graph of operations, where $G_{ij} = k$ means that the k-th operation o_k is to be placed between nodes i and j

The architecture is obtained by assembling operations $\mathbf{o}$ according to the adjacency matrix G:

$$arch = assemble(G, \mathbf{o})$$

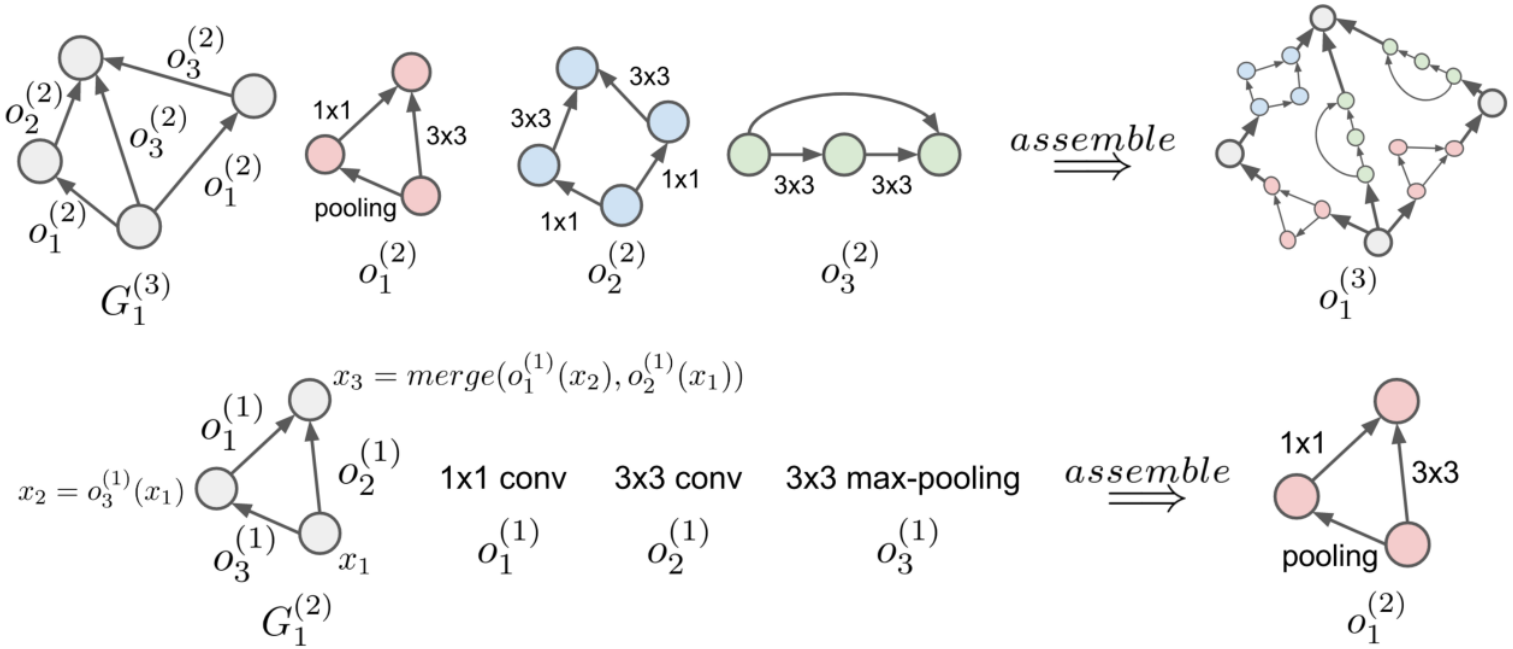


Figure 1: An example of a three-level hierarchical architecture representation. The bottom row shows how level-1 primitive operations $o_1^{(1)}, o_2^{(1)}, o_3^{(1)}$ are assembled into a level-2 motif $o_1^{(2)}$. The top row shows how level-2 motifs $o_1^{(2)}, o_2^{(2)}, o_3^{(2)}$ are then assembled into a level-3 motif $o_1^{(3)}$.

200 GPUs $\times$ 1.5 days

Algorithms

Algorithm 1: ASYNCEVO Asynchronous Evolution (Controller)

Input: Data queue $\mathcal{Q}$ containing initial genotypes; Memory table $\mathcal{M}$ recording evaluated genotypes and their fitness.

while *True* **do**
  if HASIDLEWORKER() **then**
    $genotype \leftarrow$ ASYNCTOURNAMENTSELECT($\mathcal{M}$)
    $genotype' \leftarrow$ MUTATE($genotype$)
    $\mathcal{Q} \leftarrow \mathcal{Q} \cup genotype'$

Algorithm 2: ASYNCEVO Asynchronous Evolution (Worker)

Input: Training set $\mathcal{T}$, validation set $\mathcal{V}$; Shared memory table $\mathcal{M}$ and data queue $\mathcal{Q}$.

while *True* **do**
  if $|\mathcal{Q}| > 0$ **then**
    $genotype \leftarrow \mathcal{Q}$.pop()
    $arch \leftarrow$ ASSEMBLE($genotype$)
    $model \leftarrow$ TRAIN($arch, \mathcal{T}$)
    $fitness \leftarrow$ EVALUATE($model, \mathcal{V}$)
    $\mathcal{M} \leftarrow \mathcal{M} \cup (genotype, fitness)$

Illustration of PNAS

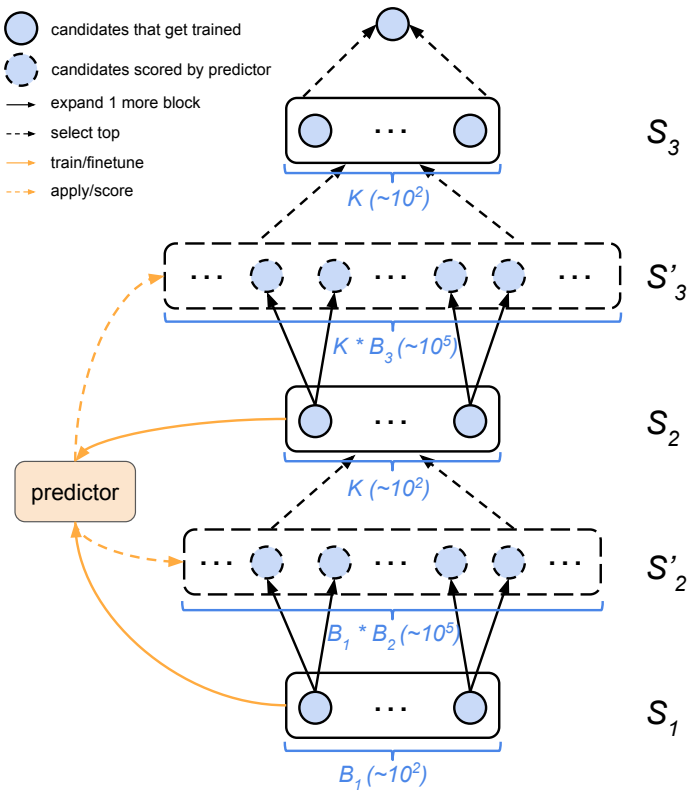
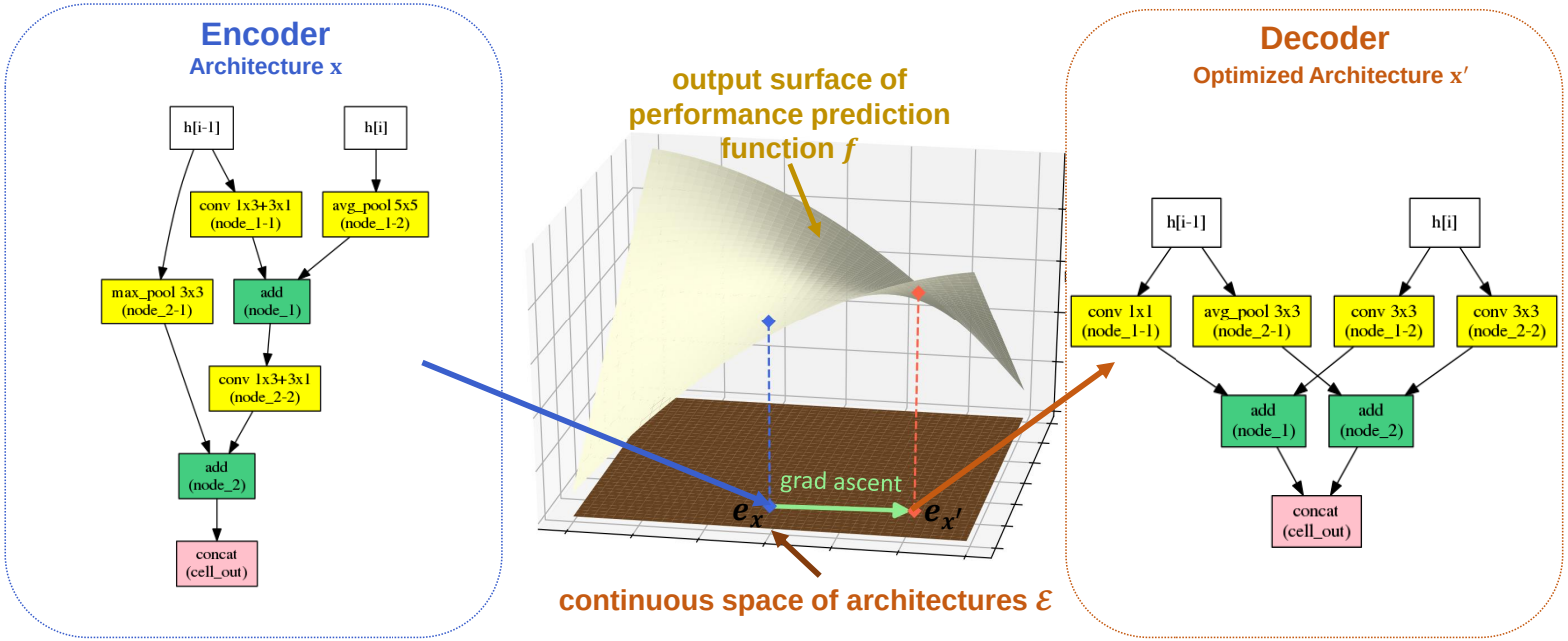


Fig. 1. Illustration of the PNAS search procedure when the maximum number of blocks is $B = 3$. Here $\mathcal{S}_b$ represents the set of candidate cells with b blocks. We start by considering all cells with 1 block, $\mathcal{S}_1 = \mathcal{B}_1$; we train and evaluate all of these cells, and update the predictor. At iteration 2, we expand each of the cells in $\mathcal{S}_1$ to get all cells with 2 blocks, $\mathcal{S}'_2 = \mathcal{B}_{1:2}$; we predict their scores, pick the top K to get $\mathcal{S}_2$, train and evaluate them, and update the predictor. At iteration 3, we expand each of the cells in $\mathcal{S}_2$, to get a subset of cells with 3 blocks, $\mathcal{S}'_3 \subseteq \mathcal{B}_{1:3}$; we predict their scores, pick the top K to get $\mathcal{S}_3$, train and evaluate them, and return the winner. $B_b = |\mathcal{B}_b|$ is the number of possible blocks at level b and K is the beam size (number of models we train and evaluate per level of the search tree).

225 GPU days

The Overall Framework of NAO



- The original architecture x is mapped to continuous representation e_x via encoder network
- e_x is optimized into $e_{x'}$ via maximizing the output of performance predictor f using gradient ascent
- $e_{x'}$ is transformed into a new architecture x' using the decoder network

Detailed Algorithm

Algorithm 1 Neural Architecture Optimization

Input: Initial candidate architectures set X to train NAO model. Initial architectures set to be evaluated denoted as $X_{eval} = X$. Performances of architectures $S = \emptyset$. Number of seed architectures K. Step size η. Number of optimization iterations L.

for $l = 1, \cdots, L$ **do**

Train each architecture $x \in X_{eval}$ and evaluate it to obtain the dev set performances $S_{eval} = \{s_x\}, \forall x \in X_{eval}$. Enlarge S: $S = S \bigcup S_{eval}$.

Train encoder E, performance predictor f and decoder D by minimizing Eqn.(1), using X and S.

Pick K architectures with top K performances among X, forming the set of seed architectures X_{seed}.

For $x \in X_{seed}$, obtain a better representation $e_{x'}$ from $e_{x'}$ using Eqn. (2), based on encoder E and performance predictor f. Denote the set of enhanced representations as $E' = \{e_{x'}\}$.

Decode each x' from $e_{x'}$ using decoder, set X_{eval} as the set of new architectures decoded out: $X_{eval} = \{D(e_{x'}), \forall e_{x'} \in E'\}$. Enlarge X as $X = X \bigcup X_{eval}$.

end for

Output: The architecture within X with the best performance

200 GPUs $\times$ 1 day & 0.4 GPU day (+weight sharing)

Outline

1. Background
2. Related Work
3. Motivation

Key Issues in NAS

- Design a high precision performance predictor
- Construct efficient architecture space
- Accelerate search process and improve sampling efficiency
- Develop multiple objective/reward

Google Cloud AutoML

- https://cloud.google.com/automl/
- Vision, Natural Language and Translation

Thank you for your attention!